

ABC Solutions Company Website

Project Overview

The ABC Solutions Company Website is a full-stack web application developed to provide information about the company's services, career opportunities, and customer communication channels. The website allows visitors to explore company solutions, view job openings, submit contact inquiries, and apply for available positions.

The application uses a modern web technology stack consisting of HTML, CSS, JavaScript, Node.js, Express.js, MongoDB Atlas, GitHub, and Vercel for deployment.

Technology Stack

Frontend

- HTML5
- CSS3
- JavaScript

Backend

- Node.js
- Express.js

Database

- MongoDB Atlas (Cloud Database)

Deployment

- GitHub
- Vercel

Live Application

Website URL

<https://abc-website-chi.vercel.app/>

Health Check API

<https://abc-website-chi.vercel.app/api/health>

Careers Page

<https://abc-website-chi.vercel.app/career>

Features

Home Page

- Displays company information and services.
- Allows visitors to submit contact inquiries.
- Stores submitted contact information in MongoDB Atlas.

Career Page

- Displays available job openings.
- Allows candidates to apply online.
- Stores job application details in MongoDB Atlas.

Database Integration

- MongoDB Atlas cloud database is used for data storage.
- Contact form submissions are stored in the contacts collection.
- Job applications are stored in the job applications collection.

API Services

- Health monitoring endpoint.
- Career listing endpoint.
- Solutions listing endpoint.
- Contact form submission endpoint.
- Job application submission endpoint.

MongoDB Atlas Configuration

Step 1: Create MongoDB Atlas Account

- Visit MongoDB Atlas.
- Create a free account using Email or Google.
- Select the M0 Free Tier plan.

Step 2: Create a Database Cluster

- Click Build a Database.
- Select M0 Free Cluster.
- Choose a cloud provider and nearest region.
- Create the cluster.

Step 3: Create Database User

- Navigate to Database Access.
- Create a new database user.
- Configure username and password.
- Assign Read and Write permissions.

Step 4: Configure Network Access

- Open Network Access.
- Add IP Address.
- Allow access from anywhere using: 0.0.0.0/0

This enables Vercel servers to communicate with MongoDB Atlas.

Step 5: Obtain Connection String

Example MongoDB URI:

mongodb+srv://username@cluster.mongodb.net/abcsolutions?retryWrites=true&w=majority

Store the URI securely and never expose it publicly.

Environment Configuration

Create a .env file in the root directory.

Example:

MONGODB_URI=your_mongodb_connection_string

NODE_ENV=development

Running the Project Locally

Install Dependencies

npm install

Build Project

npm run build

Start Development Server

npm run dev

Open the application at:

<http://localhost:5000>

API Endpoints

Endpoint	Method	Description
/api/health	GET	Check server and database status
/api/careers	GET	Retrieve available job openings
/api/solutions	GET	Retrieve company solutions
/api/contact	POST	Submit contact form
/api/careers/apply	POST	Submit job application

Testing the Application

Health Check

Visit:

<http://localhost:5000/api/health>

Expected Response:

```
{  
  "status": "ok",  
  "database": "connected"  
}
```

Contact Form Testing

Submit a contact form from the Home page and verify that the record is stored in the MongoDB database.

Job Application Testing

Submit an application through the Career page and verify that the data is saved in the database.

Deployment Process

Step 1: Push Project to GitHub

```
git init
```

```
git add .
```

```
git commit -m "Initial Commit - ABC Solutions Website"
```

```
git branch -M main
```

```
git remote add origin <repository-url>
```

```
git push -u origin main
```

Step 2: Deploy Using Vercel

1. Login to Vercel.
2. Import the GitHub repository.
3. Configure environment variables:

Variable	Value
----------	-------

MONGODB_URI	MongoDB Atlas Connection String
-------------	---------------------------------

NODE_ENV	production
----------	------------

4. Click **Deploy**.

Production Verification

After deployment:

1. Open the website URL.
2. Verify Health API response.
3. Submit a contact form.
4. Submit a job application.
5. Check MongoDB Atlas collections for saved records.

Forms Implemented

Contact Form

Location:

- Home Page

Purpose:

- Collect customer inquiries and feedback.

Database Collection:

- contacts

Career Application Form

Location:

- Career Page

Purpose:

- Collect candidate applications for job openings.

Database Collection:

- Job applications

Database Collections

Contacts Collection

Stores contact form submissions.

Fields:

- Full Name
- Email
- Subject
- Message
- Created Date

Job Applications Collection

Stores candidate applications.

Fields:

- Candidate Name
- Email
- Phone Number
- Position Applied
- Resume Information
- Submission Date

Security Considerations

1. Never upload the .env file to GitHub.
2. Keep MongoDB credentials private.

3. Restrict MongoDB IP access in production environments.
4. Validate all form inputs before saving to the database.
5. Use HTTPS for secure communication.

Project Structure

ABC-Solutions-Website/

```
|
|
├── api/
|   ├── routes/
|   ├── controllers/
|   └── server.js
|
├── models/
|   ├── Contact.js
|   ├── Career.js
|   └── JobApplication.js
|
├── public/
|   ├── index.html
|   ├── career.html
|   ├── css/
|   └── js/
|
├── scripts/
|   └── seed.js
|
├── .env
├── package.json
├── vercel.json
└── README.md
```

Detailed Explanation of Project Structure

1. api/ Folder

This folder contains all backend code.

routes/

Defines the API endpoints that users or frontend pages can access.

Examples:

- /api/contact
- /api/careers
- /api/solutions

Purpose:

Receives requests from the frontend and forwards them to controllers.

controllers/

Contains the business logic of the application.

Examples:

- Save contact form data
- Fetch job listings
- Process job applications

Purpose:

Handles what should happen when an API request is received.

Example:

```
exports.submitContact = async (req, res) => {  
  // save contact data  
};
```

server.js

Main entry point of the backend application.

Responsibilities:

- Starts Express server
- Connects MongoDB
- Loads routes
- Handles middleware

Example:

```
app.listen(5000);
```

Without this file, the backend cannot run.

2. models/ Folder

Contains MongoDB database schemas.

A schema defines how data should be stored in MongoDB.

Contact.js

Stores contact form information.

Example Data:

```
{
  "fullName": "Krishna Sai",
  "email": "krishna@gmail.com",
  "subject": "Inquiry",
  "message": "Need more details"
}
```

Career.js

Stores available job openings.

Example:

```
{
  "title": "Frontend Developer",
  "location": "Remote",
  "experience": "1-2 Years"
}
```

JobApplication.js

Stores candidate applications.

Example:

```
{
  "name": "Krishna Sai",
  "email": "krishna@gmail.com",
  "position": "Frontend Developer"
}
```

3. public/ Folder

Contains all frontend files.

These files are directly visible to users in the browser.

index.html

Home page of the website.

Contains:

- Company introduction
- Services section
- Contact form

Accessible at:

/

career.html

Career page.

Contains:

- Job listings
- Application form

Accessible at:

/career

css/

Stores stylesheets.

Examples:

style.css

career.css

Purpose:

- Colors
- Layouts
- Responsive design
- Animations

js/

Stores frontend JavaScript files.

Examples:

main.js

career.js

Purpose:

- Form validation
- API calls
- Dynamic content loading

Example:

```
fetch('/api/careers')
```

4. scripts/ Folder

Contains utility scripts.

seed.js

Populates MongoDB with sample data.

Example:

```
npm run seed
```

It automatically inserts:

- Sample job openings
- Sample company solutions

Purpose:

- Makes testing easier
- Avoids manually entering data

5. .env

Environment variables file.

Stores sensitive information.

Example:

MONGODB_URI=mongodb+srv://...
NODE_ENV=production

Purpose:

- Database connection details
- API keys
- Secret values

Never upload this file to GitHub.

6. package.json

Node.js project configuration file.

Contains:

```
{  
  "name": "abc-solutions",  
  "version": "1.0.0"  
}
```

Also stores:

Dependencies

```
"express"  
"mongoose"  
"cors"  
"dotenv"
```

Scripts

```
"dev": "nodemon server.js"  
"start": "node server.js"
```

Purpose:

- Manages packages
- Runs project commands

7. vercel.json

Configuration file for deployment on Vercel.

Example:

```
{  
  "routes": [  
    {  
      "src": "/api/*",  
      "dest": "/api/*",  
      "methods": ["GET", "POST", "PUT", "DELETE"]  
    }  
  ]  
}
```

```
    "src": "/(.*)",  
    "dest": "/api/server.js"  
  }  
]  
}
```

Purpose:

- Defines routing
- Controls deployment behavior
- Connects frontend and backend

8. README.md

Project documentation file.

Displayed on GitHub homepage.

Contains:

- Project description
- Installation steps
- Deployment guide
- API documentation
- Screenshots (optional)

Purpose:

- Helps others understand and run the project

Conclusion

The ABC Solutions Company Website successfully demonstrates a complete full-stack web application integrating frontend technologies, backend APIs, cloud database services, and cloud deployment platforms. The project provides an efficient solution for company information management, customer communication, and recruitment processes while showcasing modern web development practices.